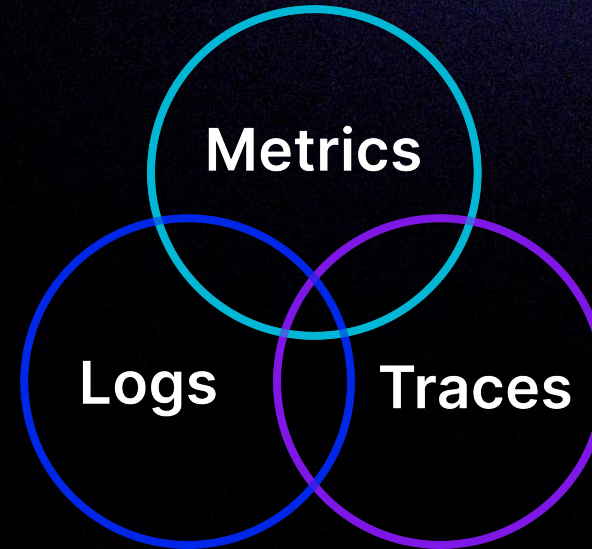


Observability 101



Jirayut Nimsaeng (Dear)

CEO & Founder, Opsta (Thailand) Co.,Ltd.

Observability Day

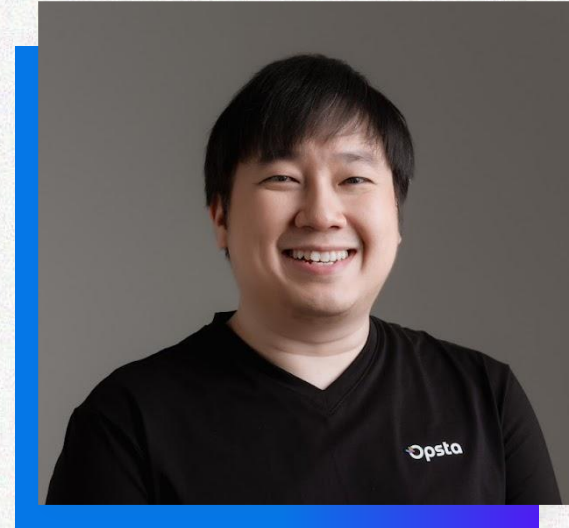


📍 **AWS Thailand Office, Singha Complex**
February 27, 2026

#whoami

Jirayut Nimsaeng (Dear)

Jirayut has been involved in DevSecOps, Container, Cloud Technology and Open Source for over 10 years. He has experienced and succeeded in transforming several companies to deliver greater values and be more agile.



- **Founder and CEO of Opsta (Thailand) Co.,Ltd.**
- **Cloud/DevSecOps Transformation Consultant and Solution Architecture**
- **First Certified Kubernetes Administrator (CKA) and Certified Kubernetes Security Specialist (CKS) in Thailand**
- **First Thai Google Cloud Developer Expert (GDE) in Thailand**
- **Google Cloud Certified - Professional Cloud Architect and Associate Cloud Engineer**

Monitoring vs Observability vs SRE

Monitoring

- Gathers information **passively**
- Focuses on **collecting** data
- **What** happened?
- How did we **fix** it last time?

Observability

- Tries to **understand** information
- Focuses on giving data **context**
- **Why** did it happen?
- How we **prevent** it in the future?

SRE

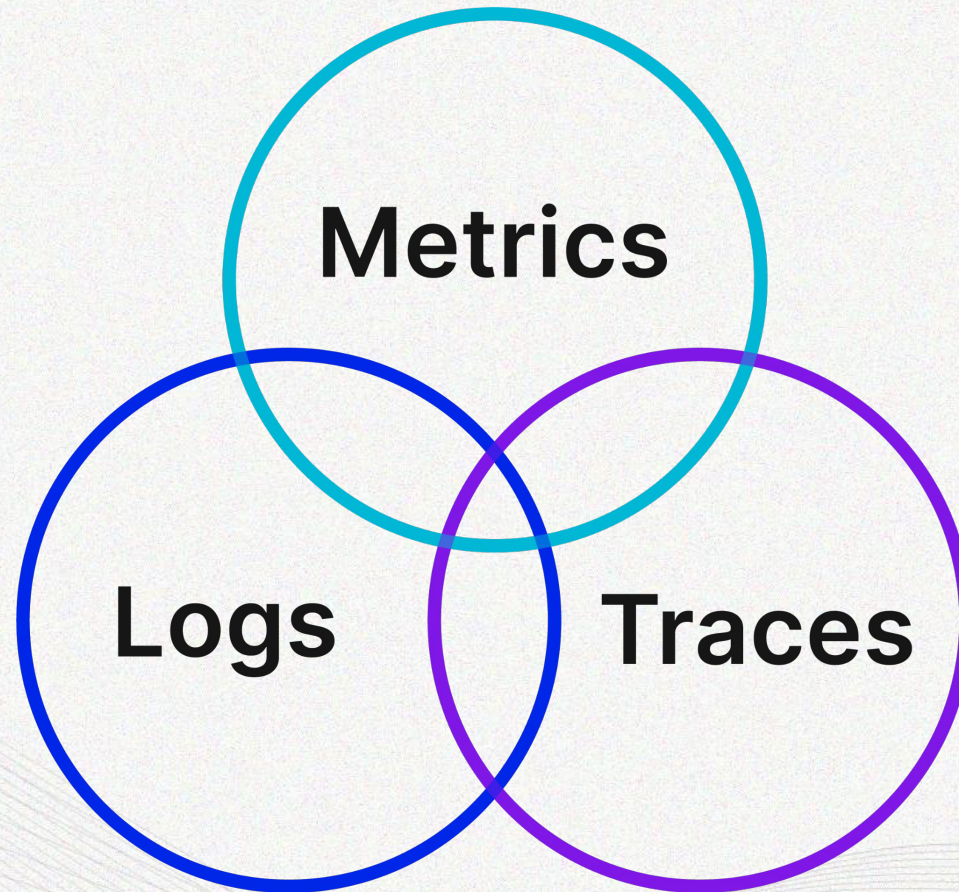
- Engineers for **reliability**
- **Balances** reliability/features
- What's our reliability **goal**?
- How to **automate** the fix?

Monitoring



What is Observability?

Three Pillars of Observability



Observability Signals

Metrics

Metrics are **numeric** representations of data. They fall into two main categories: Data that is already numeric and data distilled into numbers.



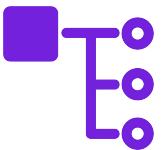
Logs

Logs are stream of **textual** entries describing usage patterns, activities, and operations within an operating system, application, server, or another device.



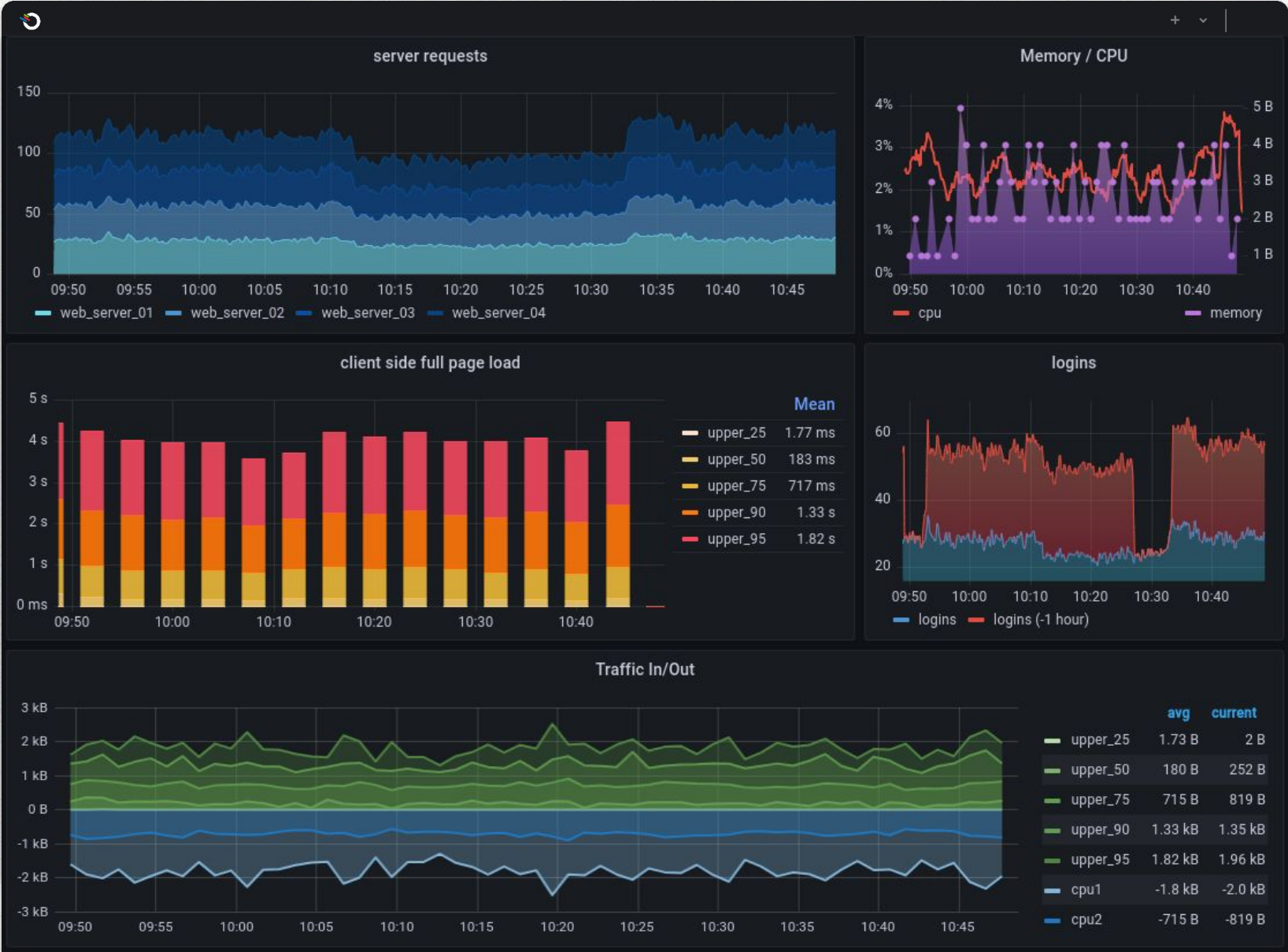
Traces

Traces are the technique of understanding what happened during a **distributed** transaction, such as a request initiated by an end-user and its effects across all downstream microservices that were touched as a result.

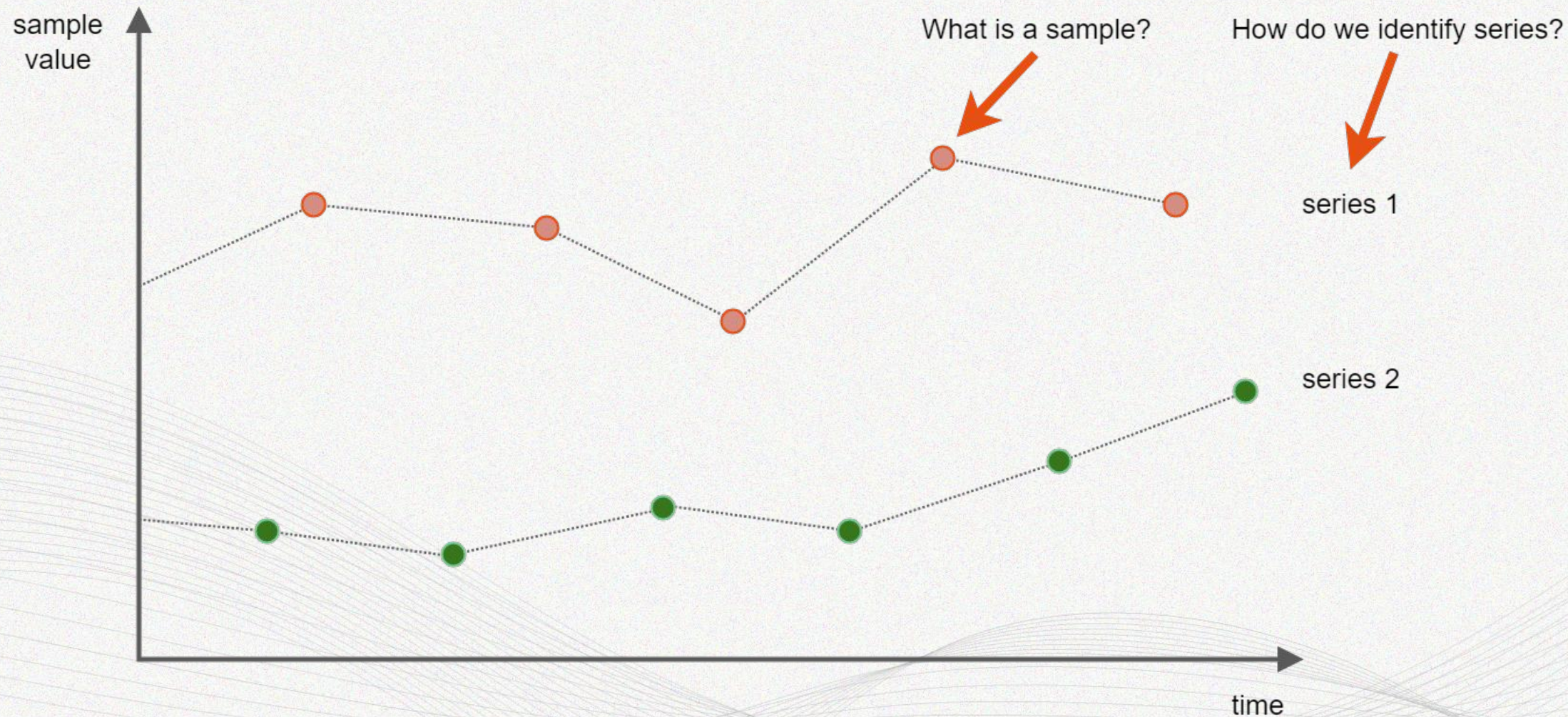


Metrics

Metrics



Time Series Data



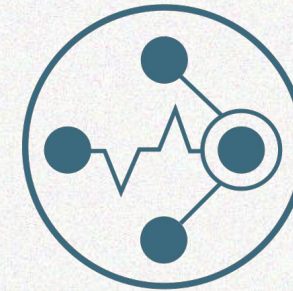
Time Series Database (TSDB)



Prometheus



Thanos



cortex



Grafana

Mimir

Time Series Data

Time Series Identifier

Samples

- 1 http_requests_total {job="nginx",instance="10.0.0.1:443"} → (t1, v1), (t2, v2), ...
- 2 http_requests_total {job="nginx",instance="10.0.0.2:443"} → (t1, v1), (t2, v2), ...
- 3 http_requests_total {job="nginx",instance="10.0.0.3:443"} → (t1, v1), (t2, v2), ...

Metric name

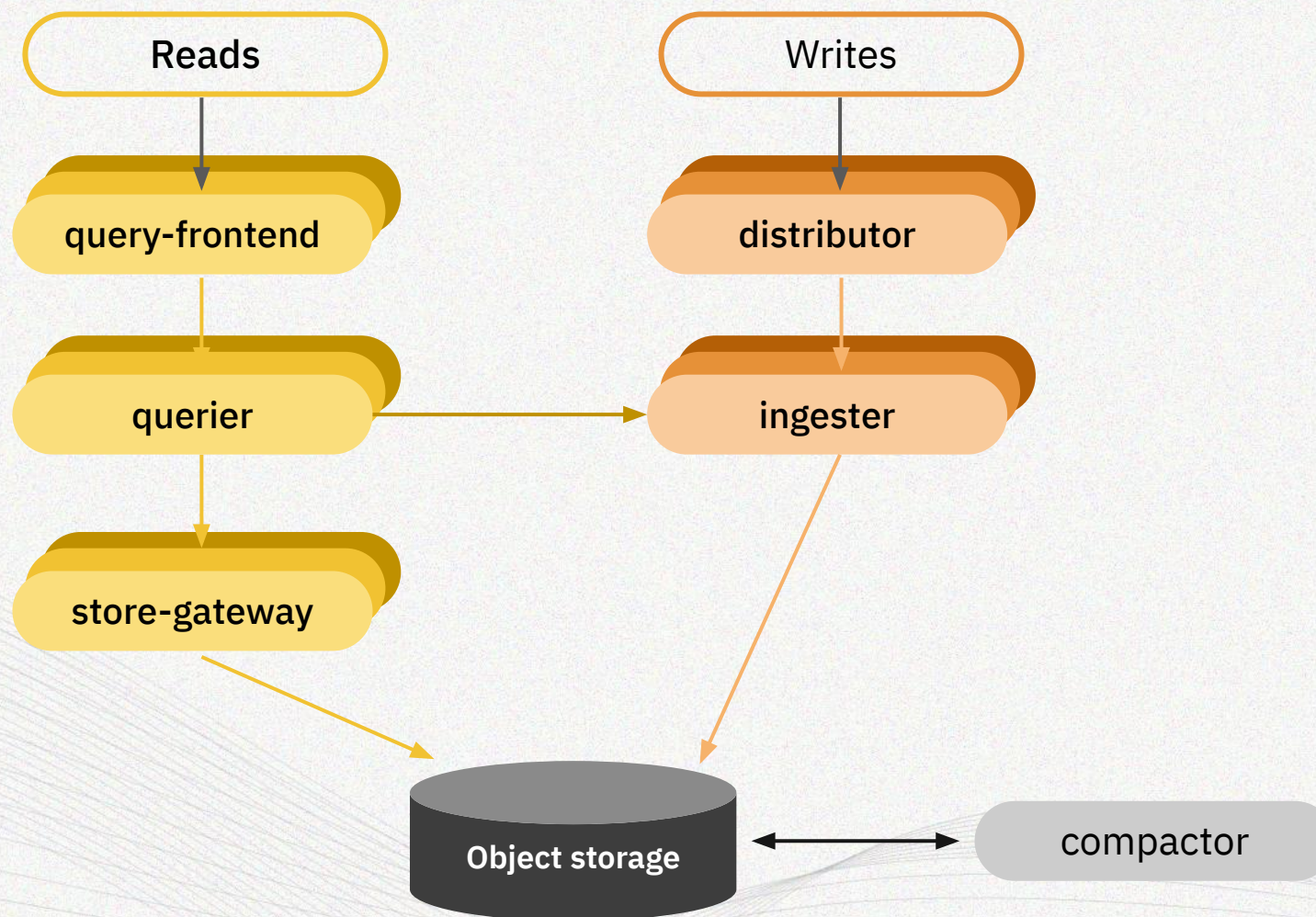
Labels

`/[a-zA-Z_:][a-zA-Z0-9_]*/`
(syntactic sugar for special label `__name__`)

label name: `/[a-zA-Z_:][a-zA-Z0-9_]*/`
label value: any UTF-8 string

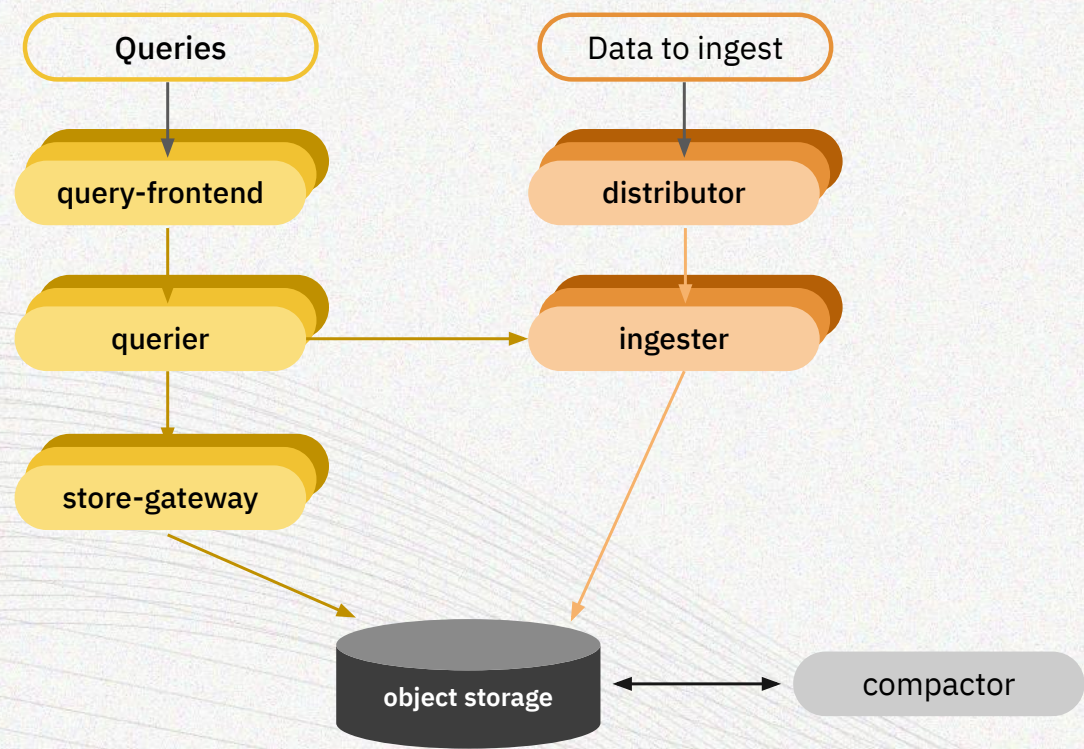
timestamps: int64 in ms
values: float64

Grafana Mimir Architecture

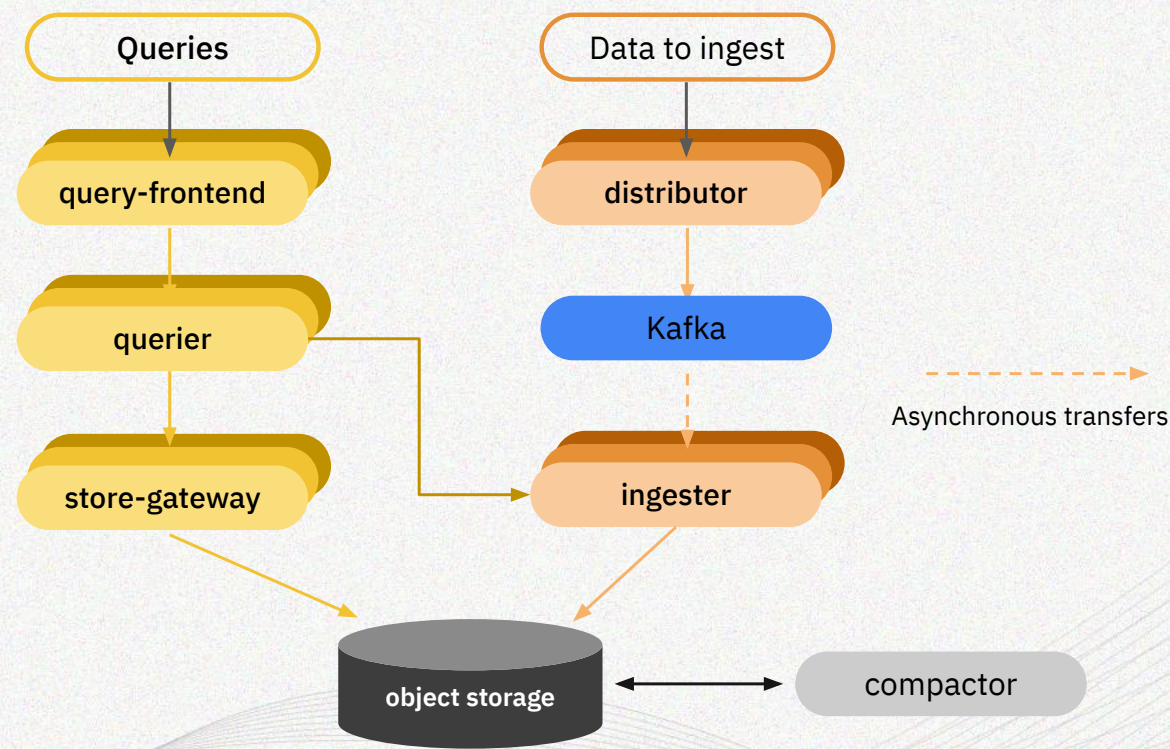


Grafana Mimir Architecture

Classic Architecture



Ingest Storage Architecture (3.0+)



Logs

Logs



Labels

service_name = loki-fundament...

Filter by label values

Data source

Loki

Last 15 minutes

Q

↺

Give feedback

Log levels

All levels

Fields

Filter by label values

Logs

248

Labels

3

Fields

87

Patterns

1

Share

Log volume (248)

75

50

25

0

15:26:00

15:27:00

15:28:00

15:29:00

15:30:00

15:31:00

15:32:00

15:33:00

15:34:00

15:35:00

15:36:00

15:37:00

15:38:00

15:39:00

15:40:00

info

Total: 247

error

Total: 1

Search in log lines

Aa

.*

Include

Exclude

Logs (248)

Newest first

Oldest first

Wrap

No wrap

Logs

Table

2025-02-21 15:40:10.116 level=info ts=2025-02-21T15:40:10.030324969Z caller=recalculate_owned_streams.go:52 msg="completed recalculate owned streams job"

Fields

caller

recalculate_owned_streams.go:52

container

loki-fundamentals-loki-1

detected_level

info

level

info

msg

completed recalculate owned streams job

service_name

loki-fundamentals-loki-1

ts

2025-02-21T15:40:10.030324969Z

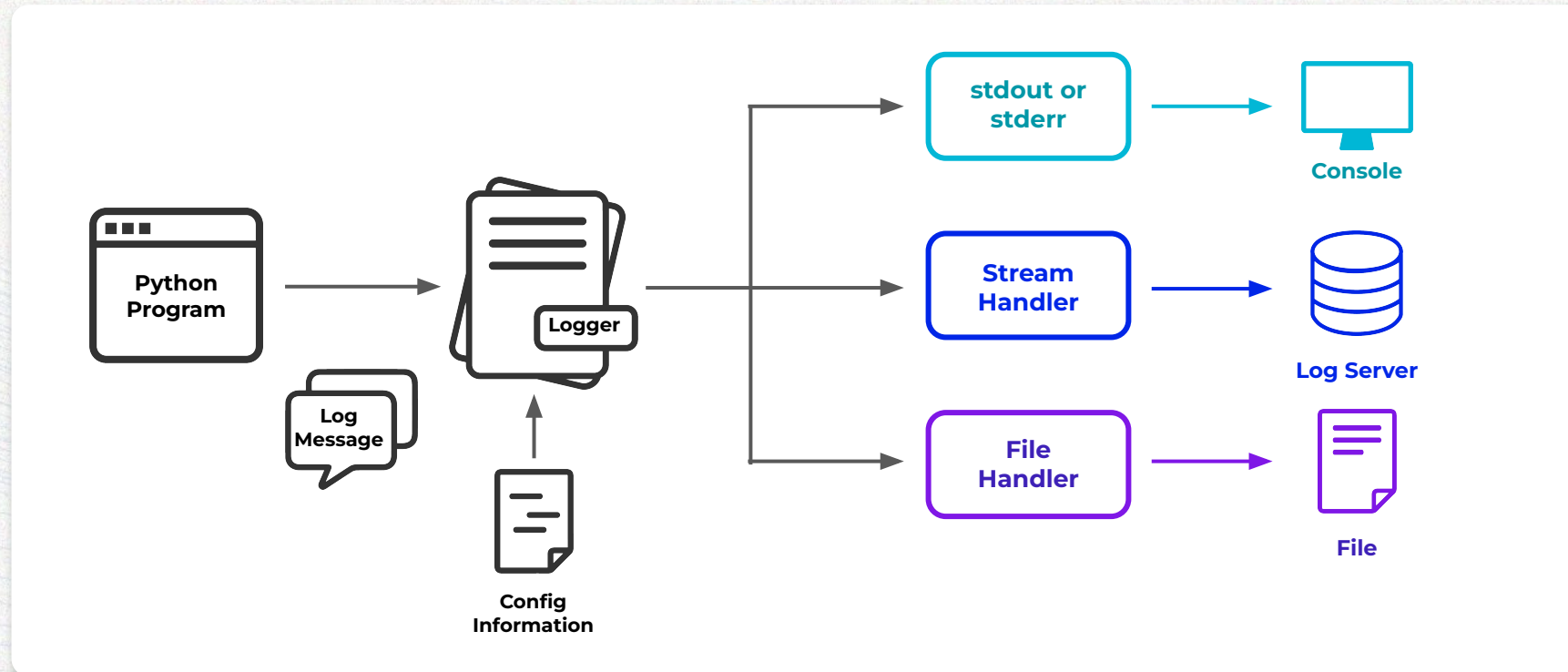
2025-02-21 15:40:10.116 level=info ts=2025-02-21T15:40:10.030127094Z caller=recalculate_owned_streams.go:49 msg="starting recalculate owned streams job"

2025-02-21 15:39:45.214 level=info ts=2025-02-21T15:39:45.201752722Z caller=marker.go:202 msg="no marks file found"

Observability 101

Logs Sources

- **Console:** output to console through STDOUT or STDERR
- **Stream:** transmit through the network and directly output the Log to the Log Server
- **File:** recorded in the file, may be used with Rotate for file management



Logs Format

Text: plain text format, usually used with Pattern to specify date, level, program name and other formats

- `%d{yyyy-MM-dd HH:mm:ss} [%-5p] [%c] [%t] %m%n`
- `2021-09-19 15:00:00 [INFO] [main] [com.example.Main] Hello World!`

Apache Access Log: Apache uses the Common Log Format (CLF) directive controls the layout and formatting of log events.

- `200.49.190.101 - - [17/May/2015:10:05:36 +0000] "GET /reset.css HTTP/1.1" 200 1015 "-" "-"`

JSON: Structured Log in JSON format

- `{"time": "2021-09-19 15:00:00", "level": "INFO", "logger": "main", "message": "Hello World!"}`

logfmt: Use the structured Log in **logfmt=** format, which is in the form of Key-Value pairs. Use pairs and spaces to separate each pair of data - represent finding graphically and identify bottlenecks

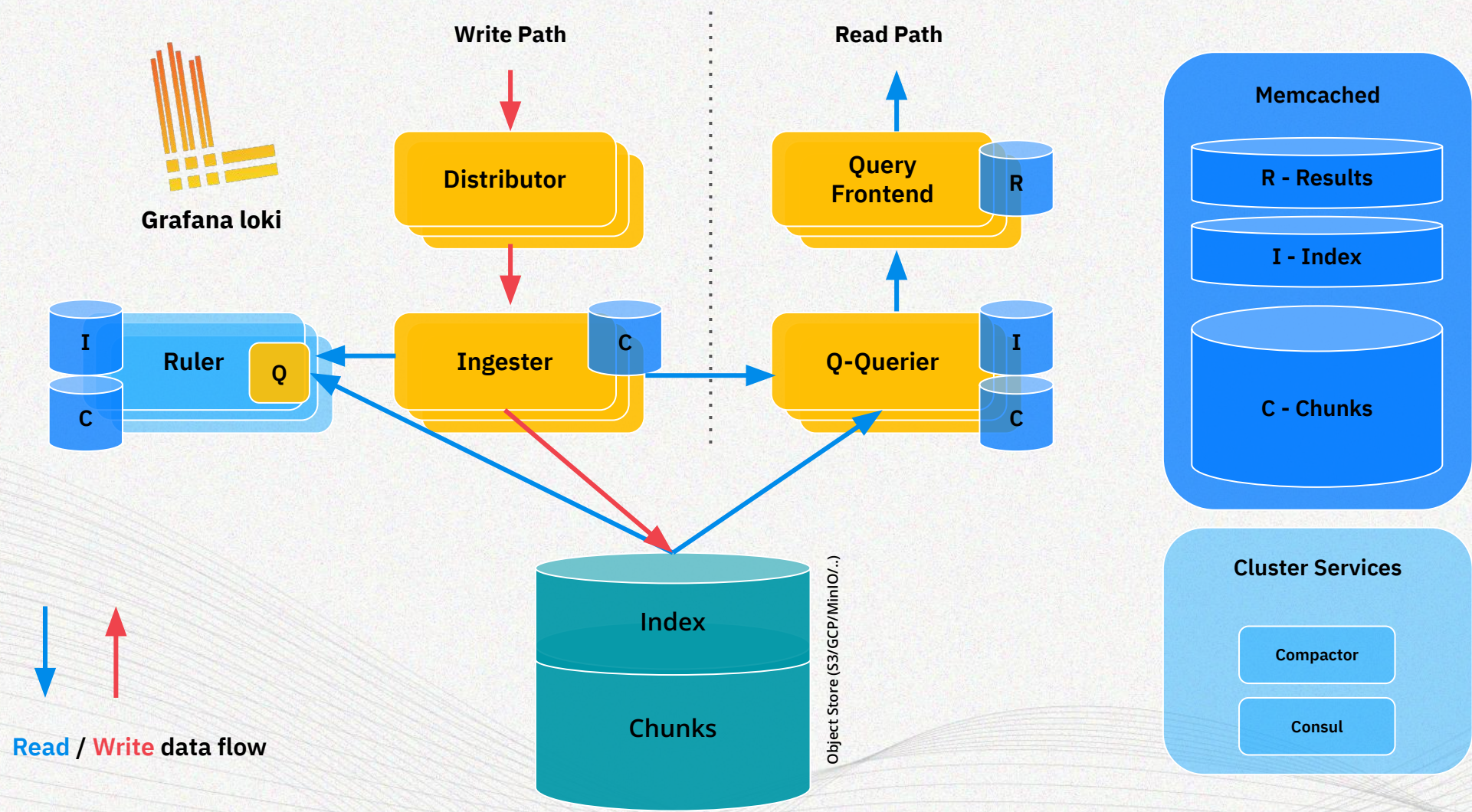
- `time="2021-09-19 15:00:00" level="INFO" logger="main" message="Hello World!"`

Prefer

Logs Format

Timestamp	Service			Level	HTTP Request			Others	
	Name	Version	Env		Path	Code	Method	Source IP	Messages
17/May/2015: 10:05:36 +0000	api	1.2.0	dev	INFO	/index.html	200	GET	200.49.190.101	User accessed '/index.html'.
17/May/2015: 10:05:36 +0000	frontend	1.1.0	uat	WARNING	/search	200	POST	200.49.190.102	Received POST data with more fields than expected. Extra fields will be ignored.
17/May/2015: 10:05:36 +0000	transaction	1.0.0	prd	ERROR	/checkout	403	POST	200.49.190.103	Failed to connect to database 'users_db', connection timed out after 30s.
17/May/2015: 10:05:36 +0000	login	1.0.0	prd	DEBUG	/login	200	GET	200.49.190.104	User 'john.doe' successfully authenticated and accessed '/dashboard'.

Grafana Loki Architecture



Grafana Loki Indexing Logs

2019-12-11T10:01:02.123456789Z

{app="nginx" , instance="1.1.1.1"}

GET /about

Timestamp
with nanosecond precision

Prometheus-style Labels
Key-value pairs

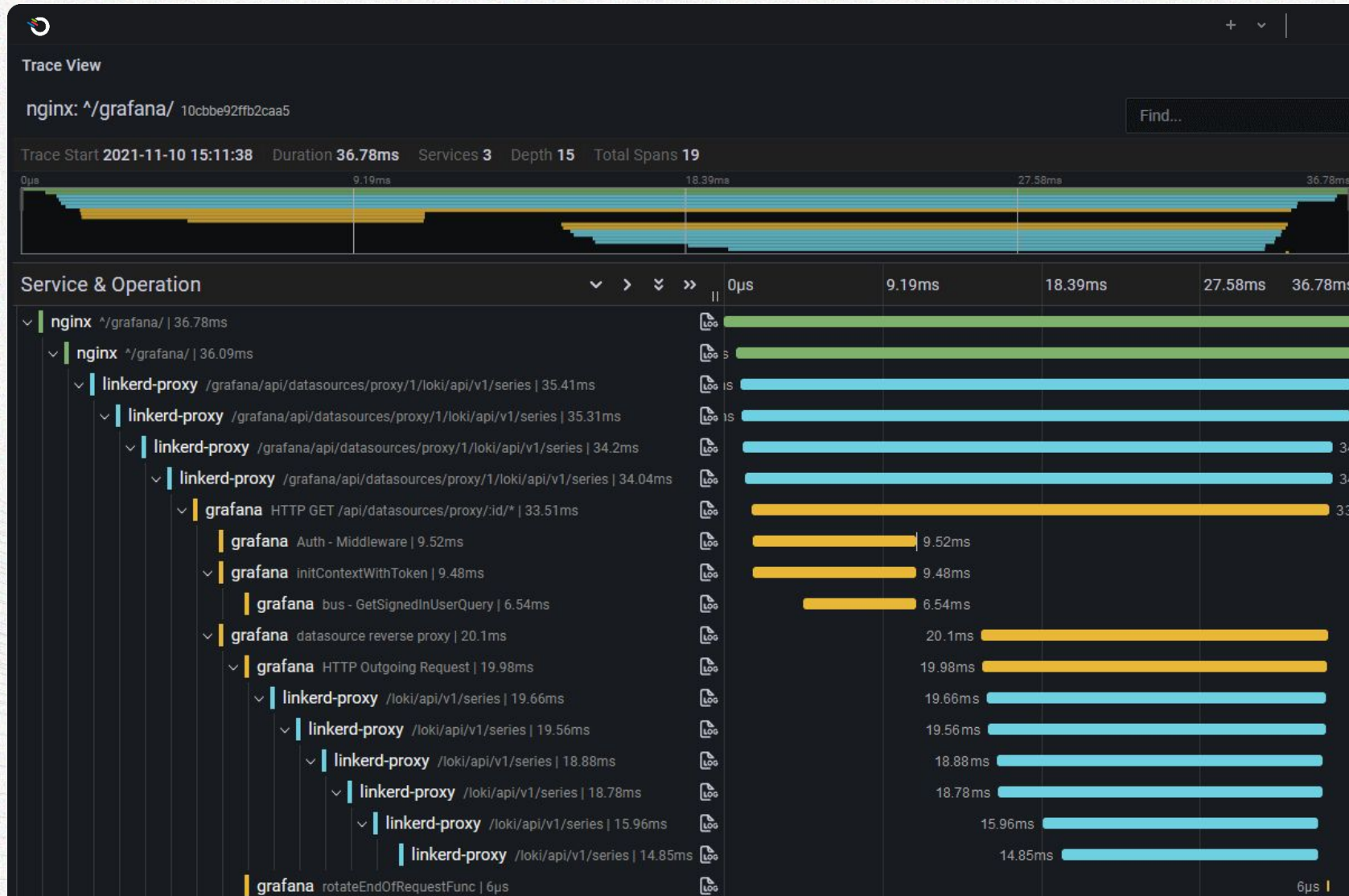
Content
log line

indexed

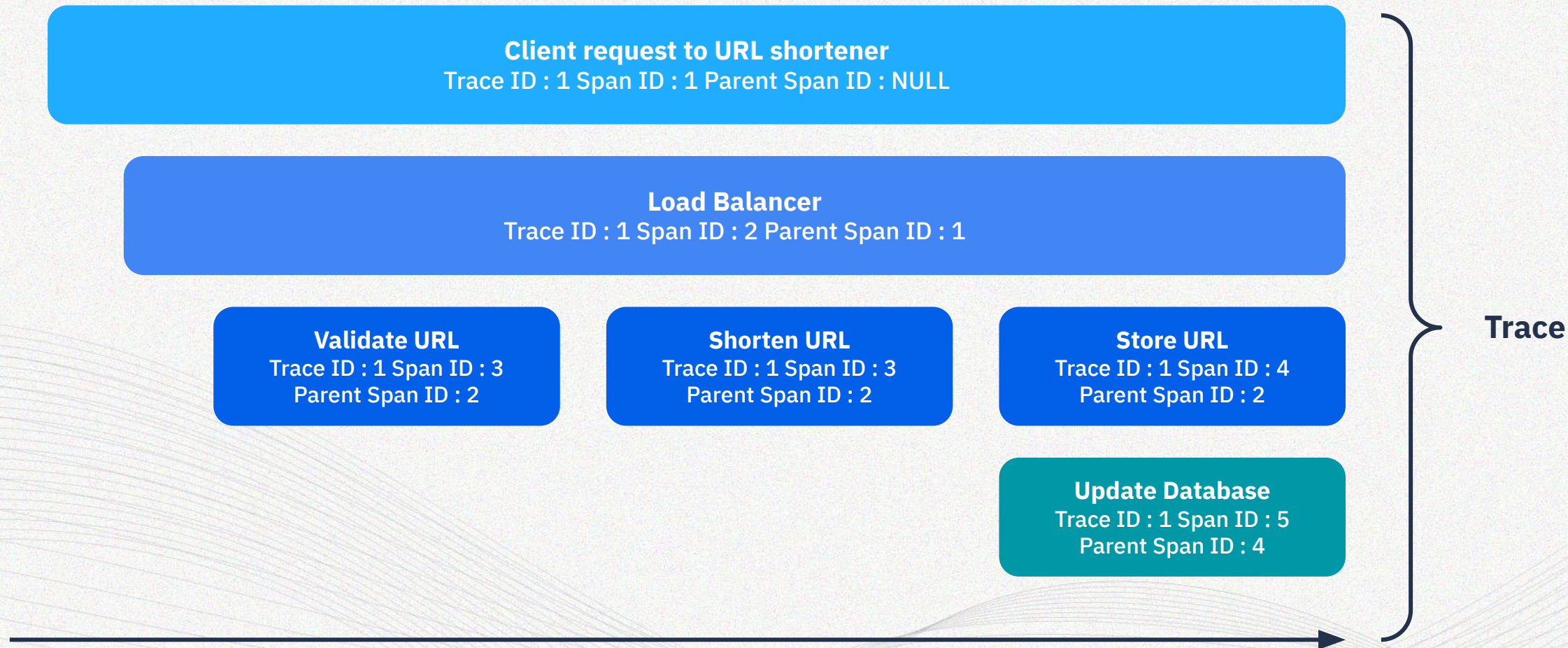
unindexed

Traces

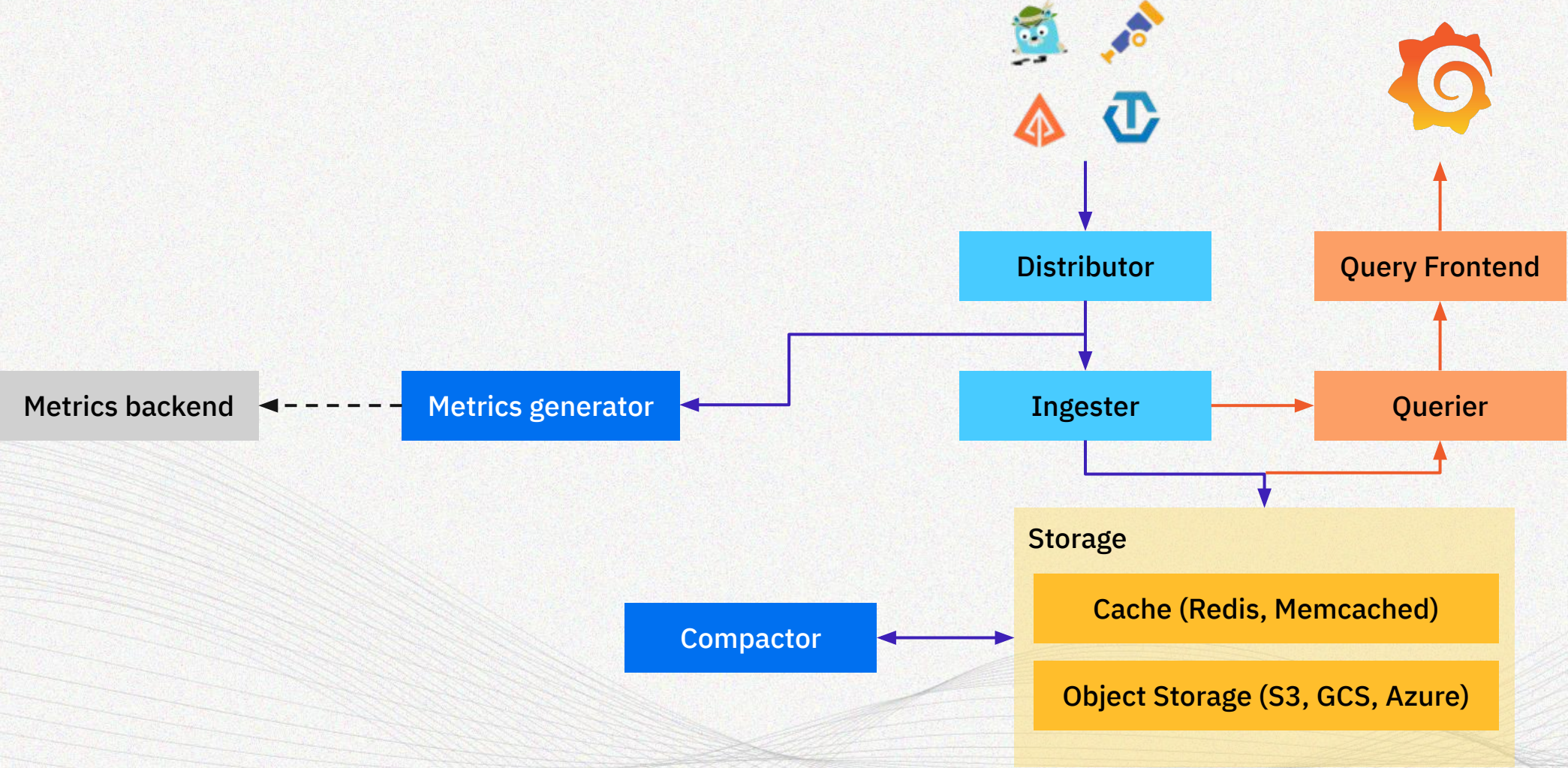
Traces



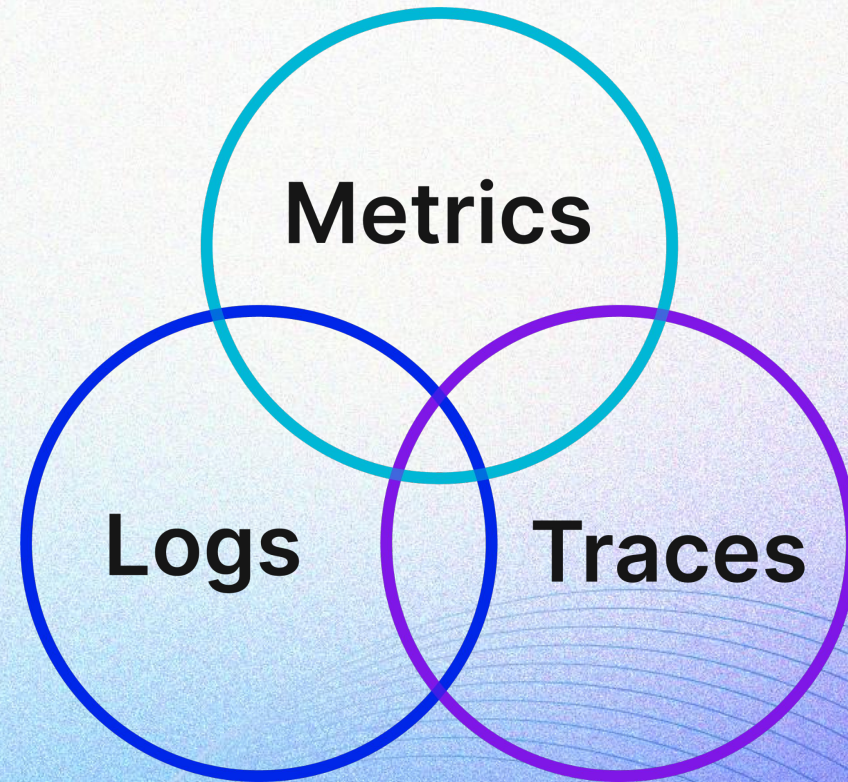
Traces and Spans



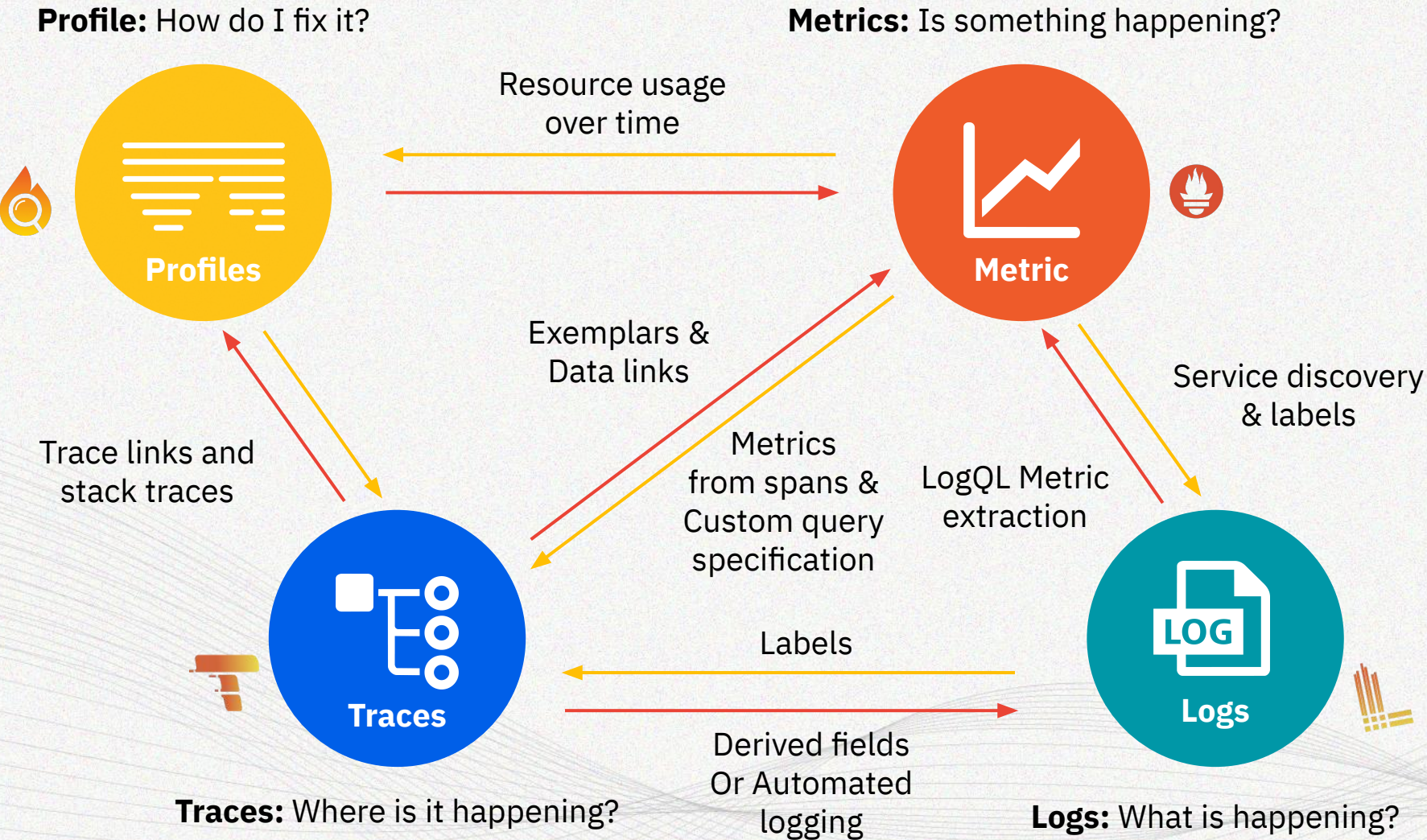
Grafana Tempo Architecture



Observability Correlation



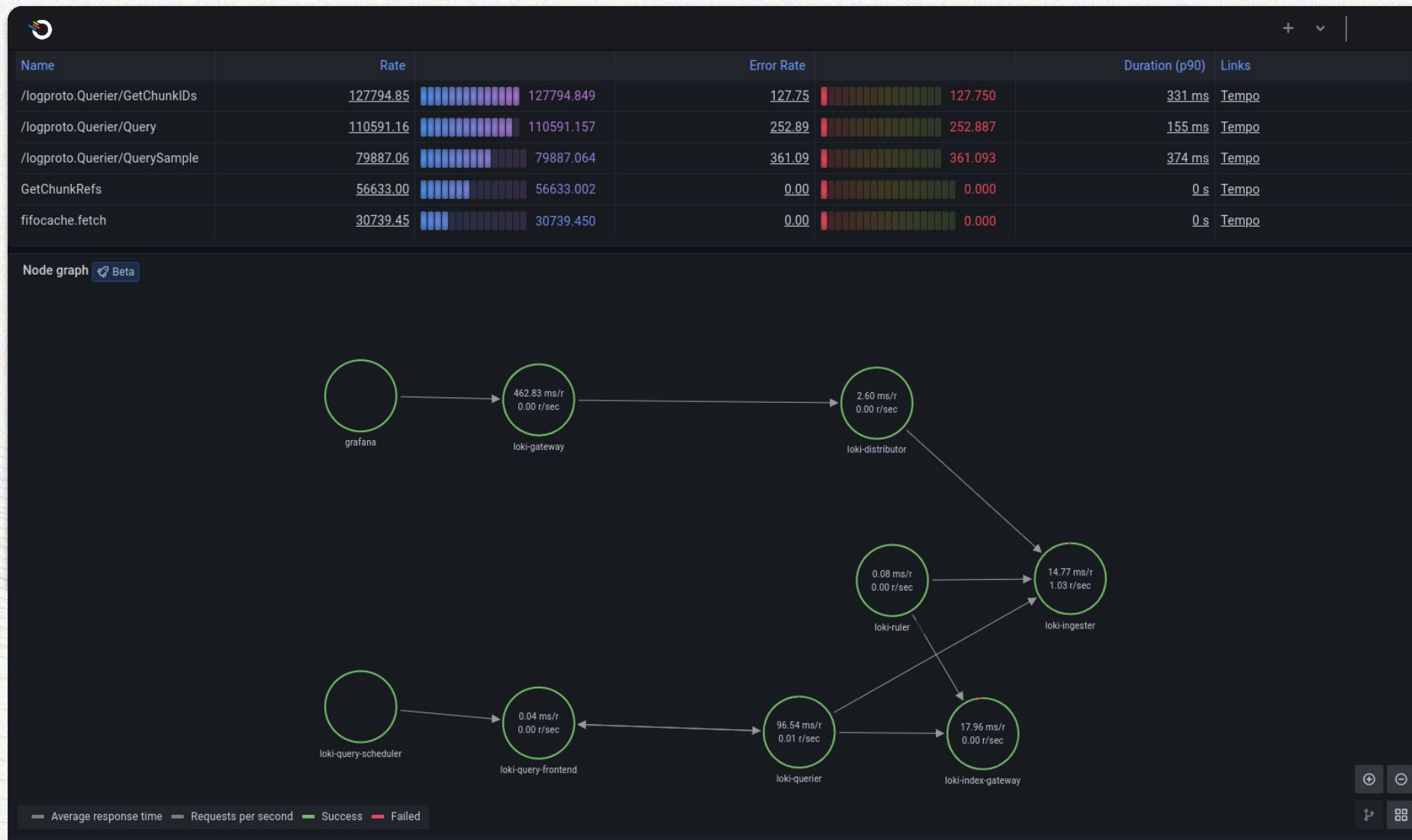
Signal Correlation



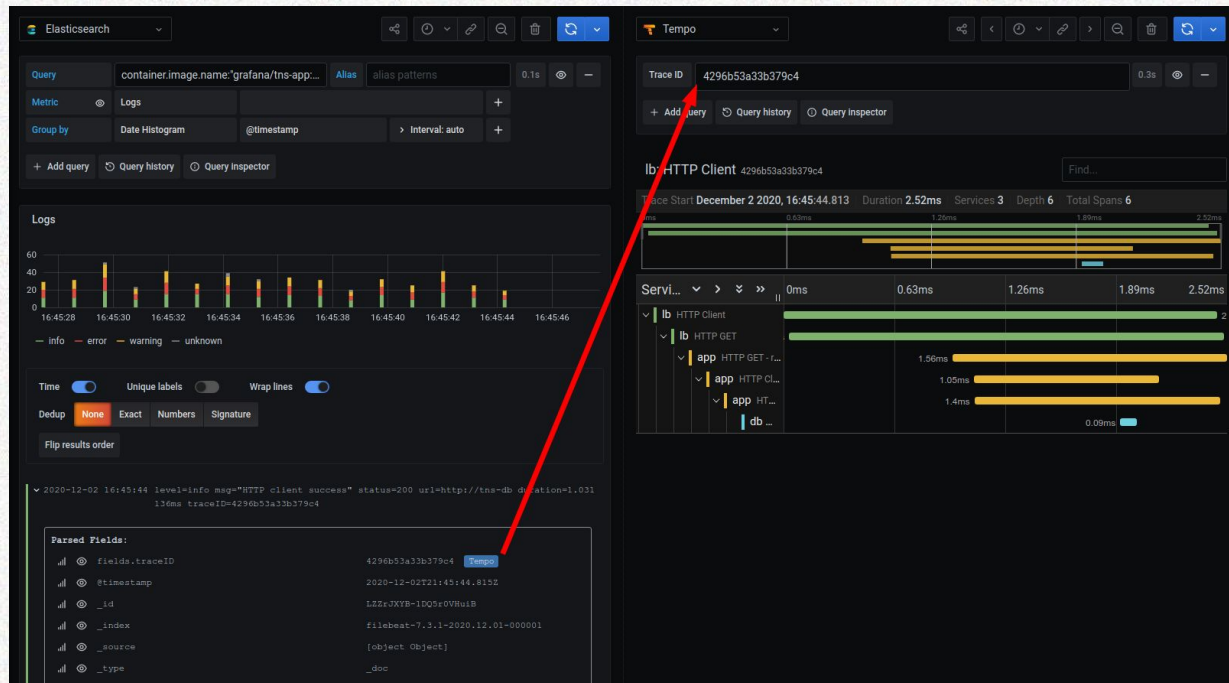
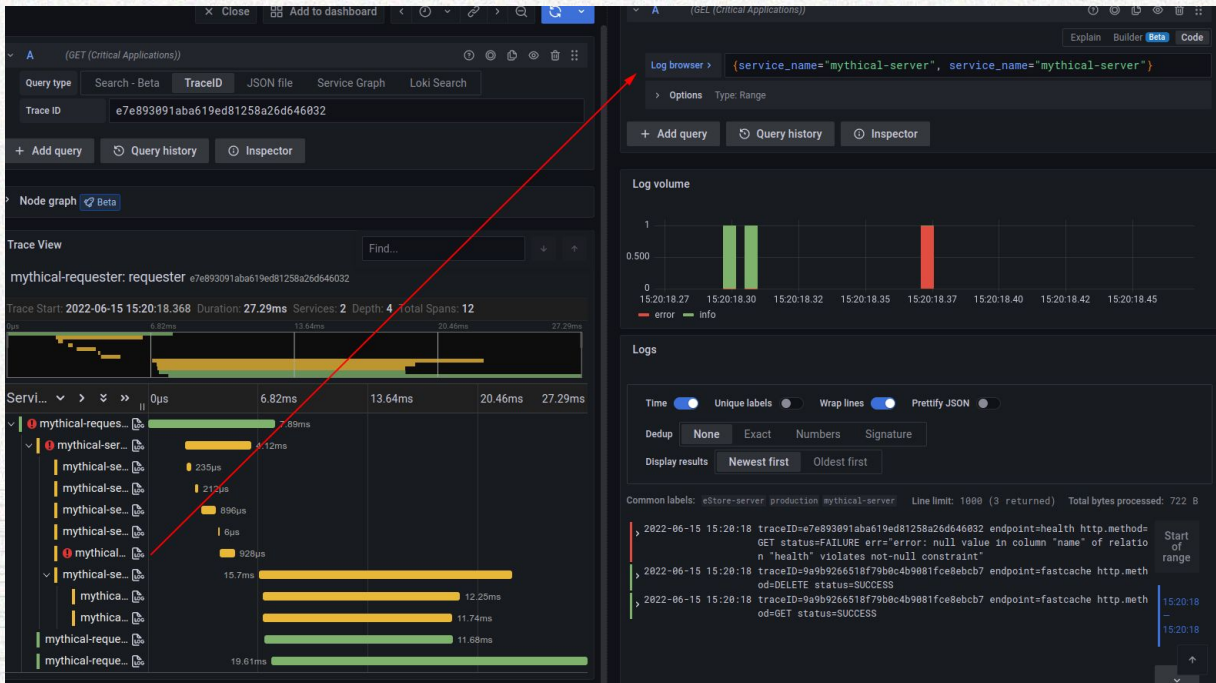
Generate Metrics from Span



Service Graph from Traces



Traces to Logs and Logs to Traces

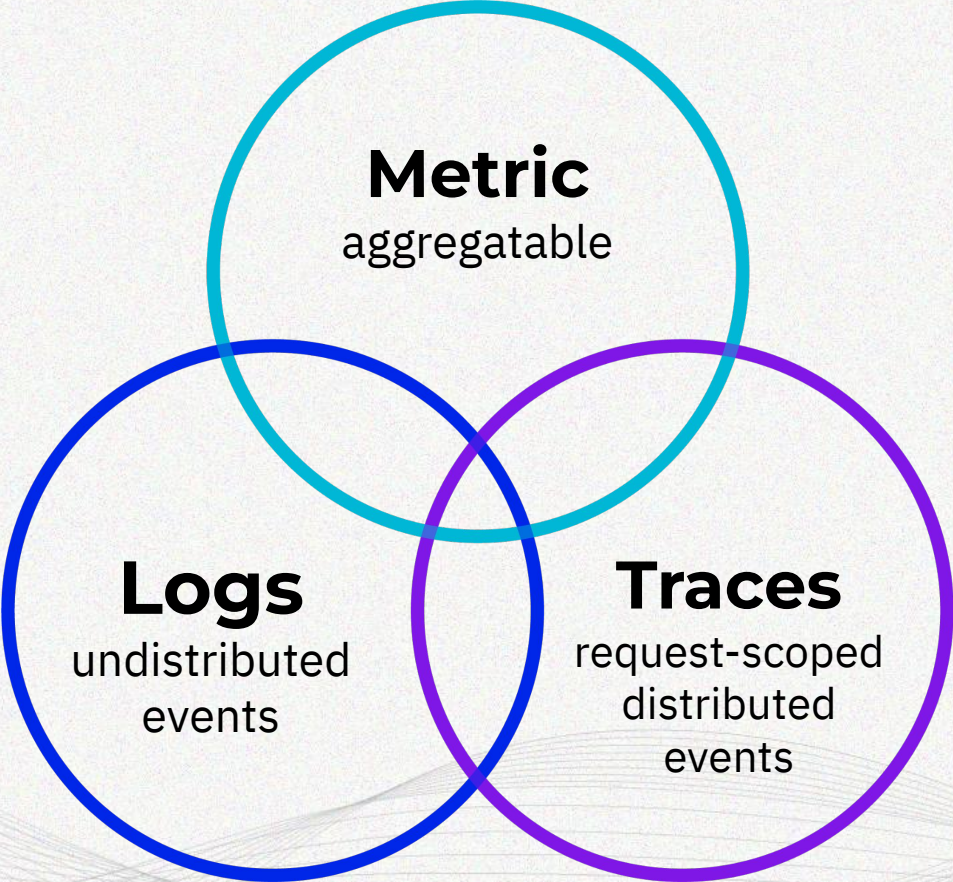


Observability Volume

Primary Signals

Lower volume

Higher volume



Collectors

Collector Software



Applications and infrastructure



Databases and collectors

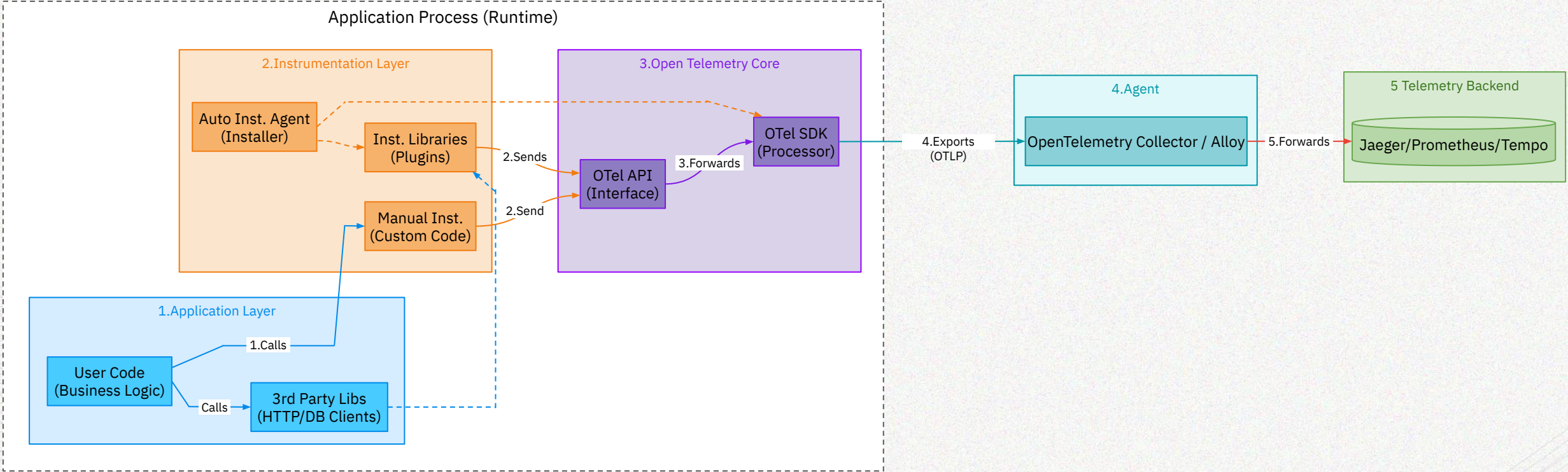


What is OpenTelemetry?

- **Observability Framework:** A unified toolkit for managing Traces, Metrics, and Logs.
- **Vendor-Agnostic:** Decouples instrumentation from specific vendors. No vendor lock-in.
- **Focused Scope:** Handles data **generation, collection, and export** only.
- **Not a Backend:** It is *not* a storage or visualization tool (like Jaeger or Prometheus).
- **Universal:** Designed to instrument any application, language, or infrastructure easily.



OpenTelemetry Instrumentation Architecture



OTEL Language APIs & SDKs Status

Language	Traces	Metrics	Logs
C++	Stable	Stable	Stable
C#/.NET	Stable	Stable	Stable
Erlang/Elixir	Stable	Development	Development
Go	Stable	Stable	Beta
Java	Stable	Stable	Stable
JavaScript	Stable	Stable	Development
Kotlin	Development	Development	Development
PHP	Stable	Stable	Stable
Python	Stable	Stable	Development
Ruby	Stable	Development	Development
Rust	Beta	Beta	Beta
Swift	Stable	Development	Development

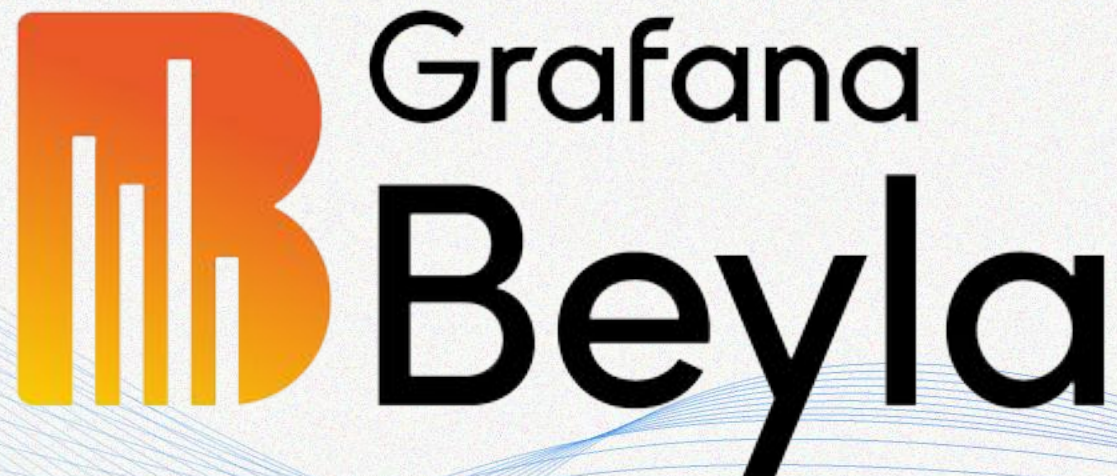
<https://opentelemetry.io/docs/languages/>



Grafana Beyla

What is Grafana Beyla?

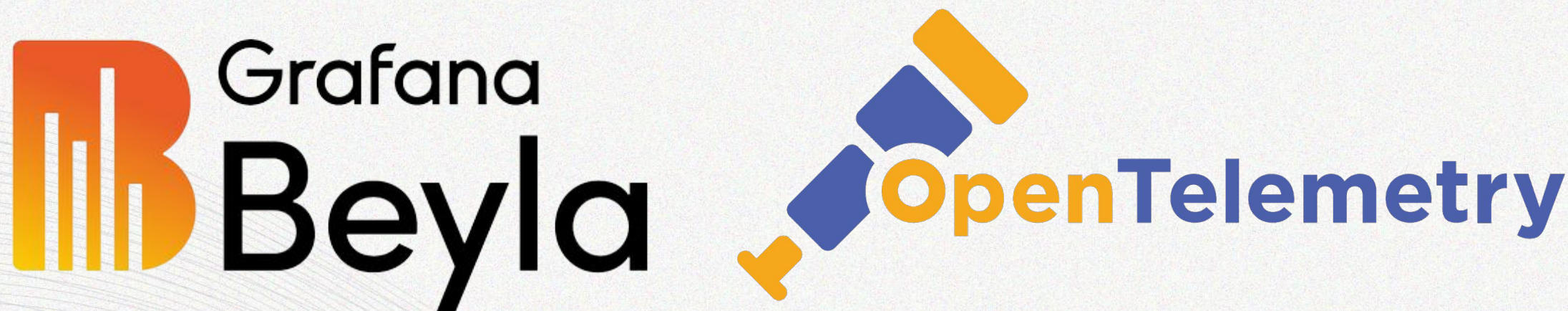
- **eBPF Auto-Instrumentation:** No code changes or language agents required
- **Zero-Touch Metrics:** Instantly captures RED metrics and traces for HTTP/S and gRPC
- **Multi-Language Support:** Works with Go, Java, Python, Rust, and Node.js
- **Open Standards:** Seamlessly exports data via OpenTelemetry and Prometheus formats.



Grafana Beyla Limitation

- **Basic Telemetry Depth:** Provides generic RED metrics and spans. Manual instrumentation is still needed for complex internal business logic.
- **Kernel Requirements:** Requires Linux Kernel 5.8 or higher with BTF enabled, which may not be available on older legacy distributions.
- **Elevated Privileges:** Needs specific Linux capabilities or root access to attach eBPF probes to the kernel and monitor network traffic.
- **Generic Tracing:** Captures transaction-level spans but cannot automatically track internal function calls without manual code intervention.

Grafana Labs donated Beyla to OpenTelemetry



<https://grafana.com/blog/opentelemetry-ebpf-instrumentation-beyla-donation/>

Visualization (Dashboards)

The USE Method

USE

U

= **Utilization:** the average time that the resource was busy servicing work

S

= **Saturation:** the degree to which the resource has extra work which it can't service, often queued.

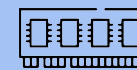
E

= **Errors:** the count of error event

CPU Usage



Memory Usage



Disk Usage



Task Queue



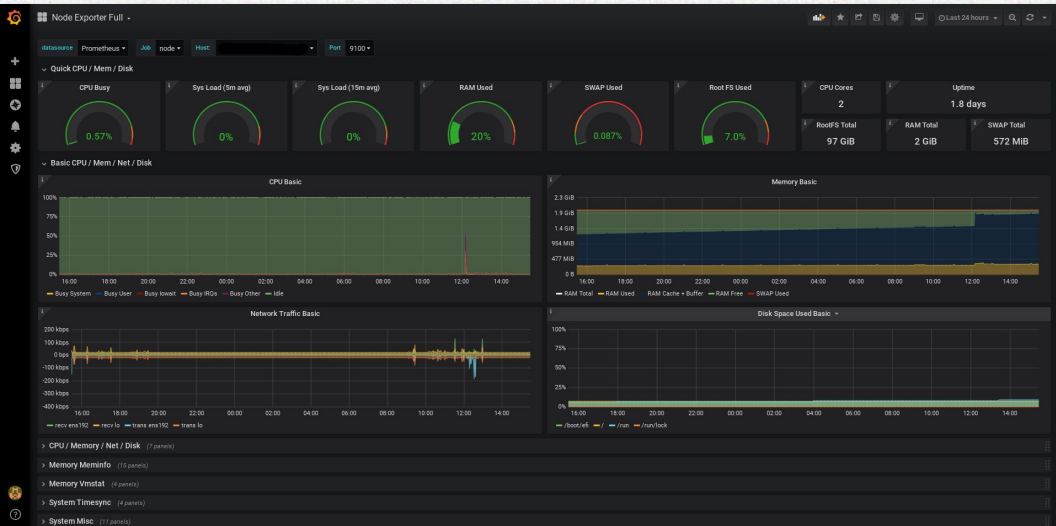
CPU Temperature



RAID Errors



Linux/Windows/Docker/Kubernetes Dashboards



Observability Dashboard: The RED Method

RED

R

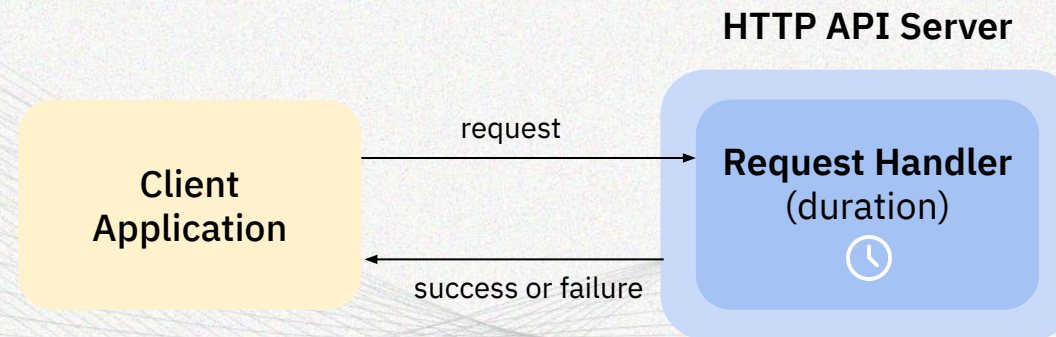
= **Request** number of requests/second, your services are serving.

E

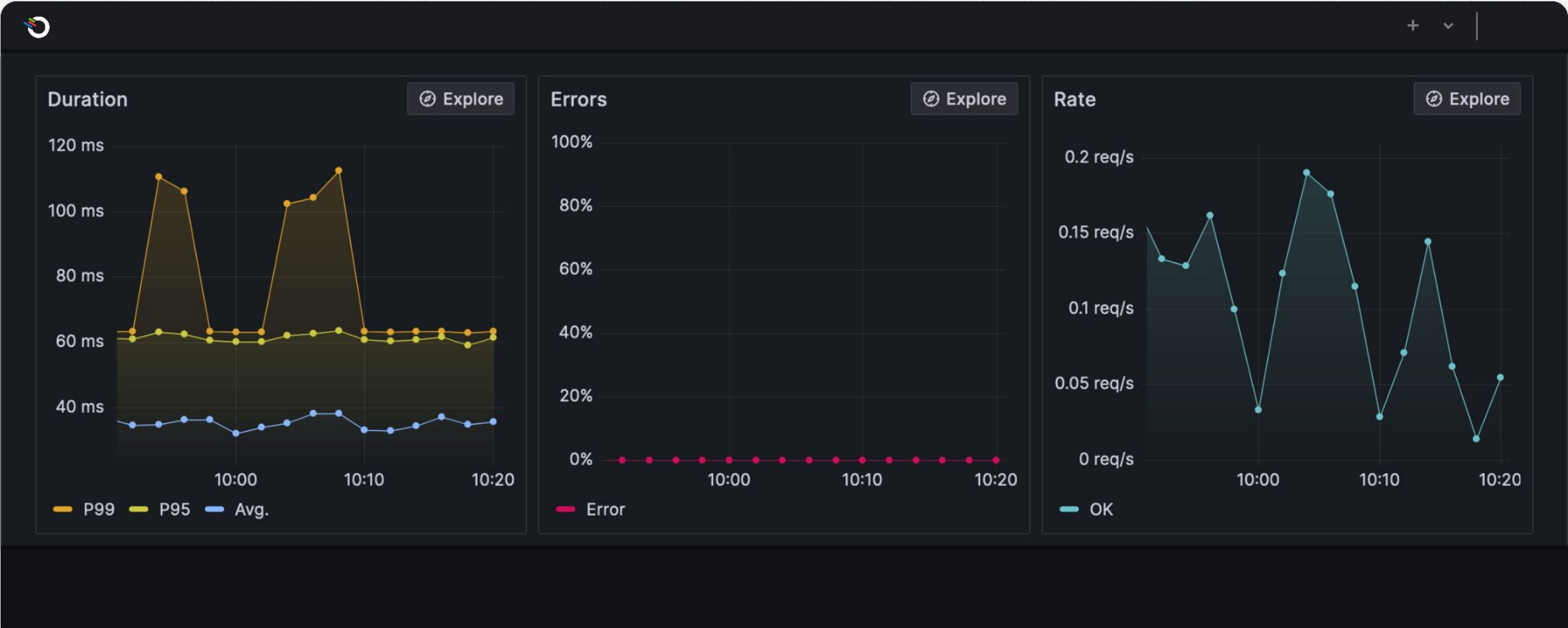
= **Errors** number of failed requests/second.

D

= **Duration** distributions of the amount of time each request takes.



RED-based Dashboard



RED-based Advanced Dashboard



Live Demo

LGTM Stack



CORE LGTM STACK



Logs

powered by Grafana Loki



Grafana

for visualization



Traces

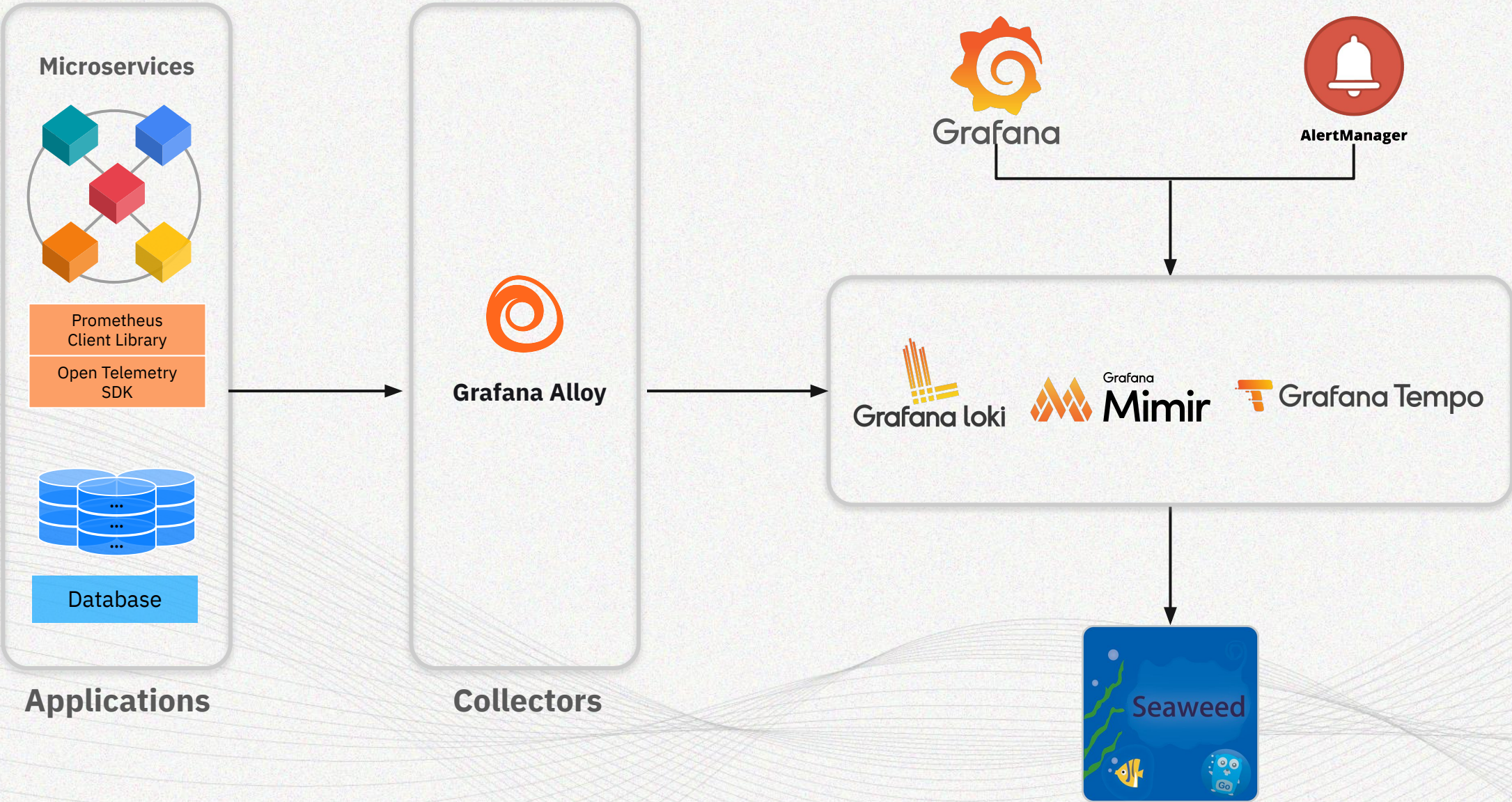
powered by Grafana Tempo



Metrics

powered by Grafana Mimir
and Prometheus

LGTM Architecture



Key Takeaway

Monitoring vs Observability vs SRE

 **Monitoring**

//

**Collecting
every data**

//

 **Observability**

//

**Understand
the data**

//

 **SRE**

//

**Make it reliable
from data**

//

Future of Observability



Observability 1.0



**Siloed
Tools**



Observability 2.0



**Signal
Correlation**



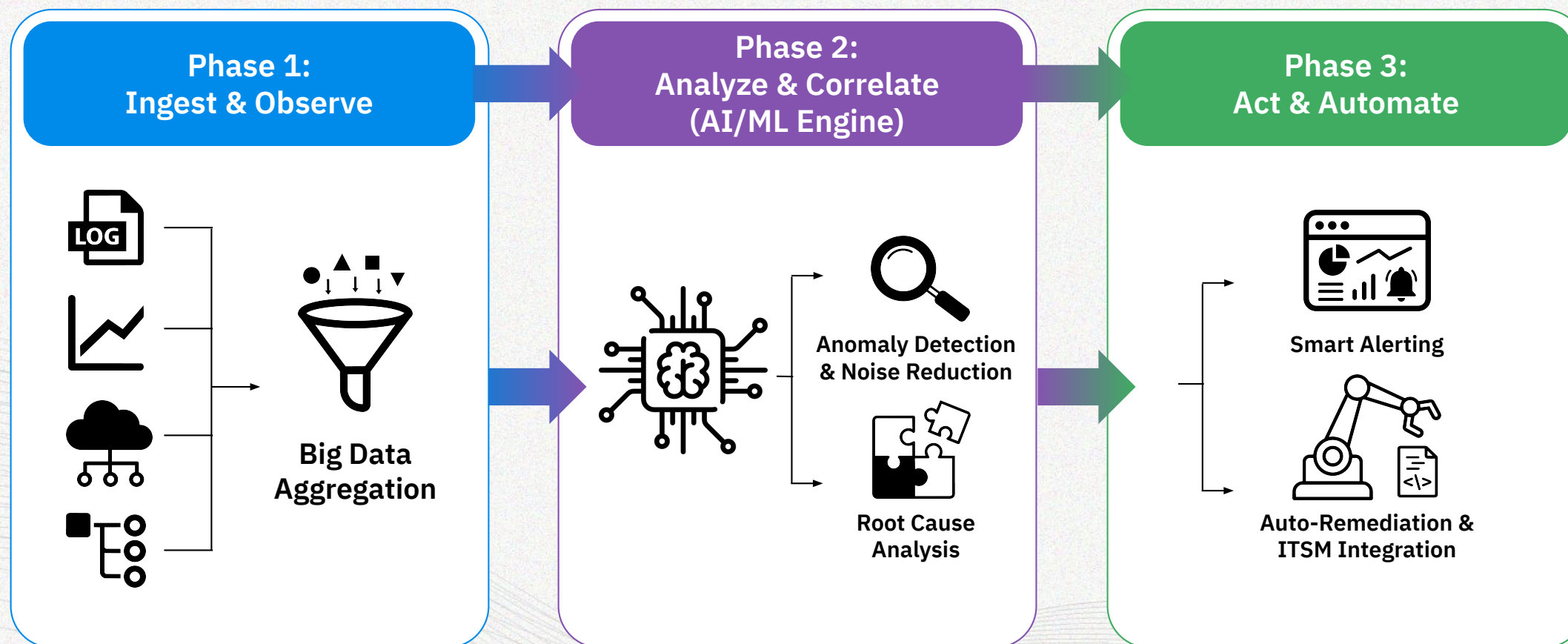
Observability 3.0



**Business
Impact**



AIOps (AI for IT Operations)



PLATFORM ENGINEERING SERIES

#3 DevSecOps Meetup 2026

Monitoring as a Centralized Observability Platform



25 MAR 2026



2:00 PM - 3:30 PM



Mr. Jirayut Nimsaeng
CEO & Founder of Opsta

JOIN WEBINAR



DevSecOps
Community
Thailand



TechTalk
THAI



SPEAKER



JIRAYUT NIMSAENG

CEO, Opsta (Thailand) Co., Ltd.

summit.fossasia.org

 March 9-10, 2026



Hall 3, True Digital Park West



10 March, 16:10

50% OFF - USE CODE

jirayut



fossasia HACKATHON

Powered by
 ExpressVPN

10 MARCH • TRUE DIGITAL PARK

Digital Safety
for Young Users

Grand Prize

THB 29,999

summit.fossasia.org



SCAN TO SIGN UP!

Contact Us



Jirayut Nimsaeng (Dear)
Founder & CEO of Opsta



fb.me/DearJirayut



www.linkedin.com/in/jirayut/



jirayut@opsta.co.th



www.opsta.co.th



<https://www.opsta.co.th>